

General Training for Programming Competitions & Coding Interviews

November 13, 2024

Heaps, Sorting and Hash Tables

Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science



Heaps

Heaps

A **heap** (or **priority queue**) is a **special binary tree** with **two properties**:

- **Shape property**: A leaf node at depth $k > 0$ can exist only if all 2^{k-1} nodes at depth $k - 1$ exist. Additionally, nodes at a partially filled level must be added "from left to right". (complete binary) The root node has a depth of 0.
- **Heap property**: The key at each node is at least as great as the keys stored at its children. (max-heap)

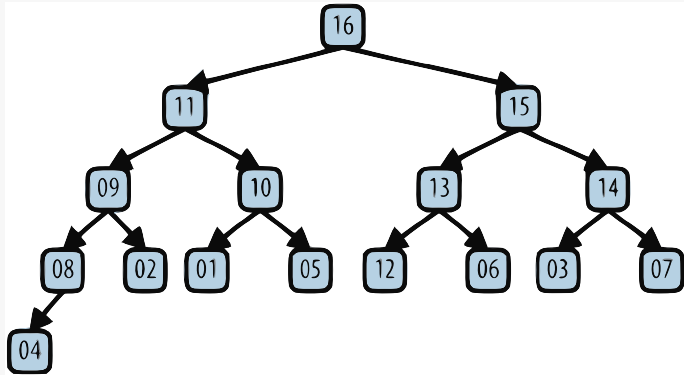
Heaps

A **heap** (or **priority queue**) is a **special binary tree** with **two properties**:

- **Shape property**: A leaf node at depth $k > 0$ can exist only if all 2^{k-1} nodes at depth $k - 1$ exist. Additionally, nodes at a partially filled level must be added "from left to right". (complete binary) The root node has a depth of 0.
- **Heap property**: The key at each node is at least as great as the keys stored at its children. (max-heap)

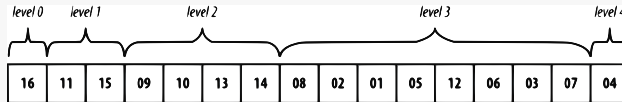
Heaps do not have the binary search tree property!

Heaps: Example



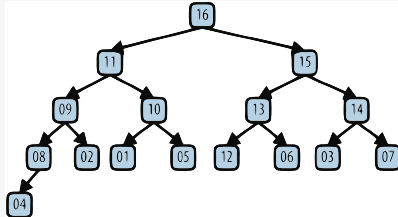
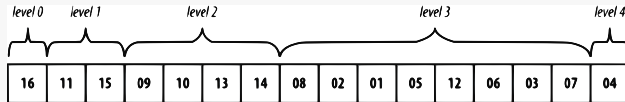
Heaps

Given the **rigid structure** imposed by the shape property, a heap can be stored in an **array** without losing any of its structural information. For a node with label i , its left child is labeled $2 * i + 1$; its right child is labeled $2 * i + 2$. Similarly, for a non-root node labeled i , its parent node is labeled $\lfloor \frac{i-1}{2} \rfloor$.



Heaps

Given the **rigid structure** imposed by the shape property, a heap can be stored in an **array** without losing any of its structural information. For a node with label i , its left child is labeled $2 * i + 1$; its right child is labeled $2 * i + 2$. Similarly, for a non-root node labeled i , its parent node is labeled $\lfloor \frac{i-1}{2} \rfloor$.

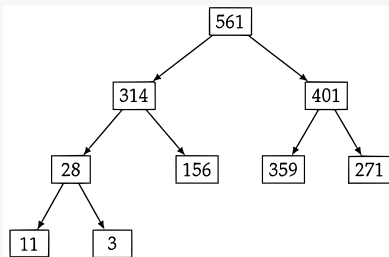


Basic Heap Operations

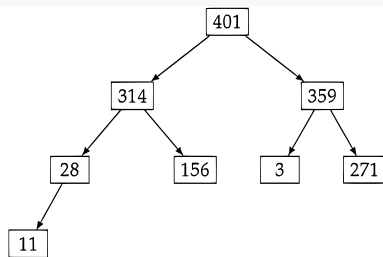
- To insert an element, add it as a child. (There is only one possible position.)
Switch the newly-added element with its parent until the heap property is regained.

Basic Heap Operations

- To insert an element, add it as a child. (There is only one possible position.)
Switch the newly-added element with its parent until the heap property is regained.
- To delete the top element, **overwrite its value** with the last leaf, and then **switch the new top element downwards** until the heap property is regained.



(a) A max-heap. Note that the root holds the maximum key, 561.



(b) After the deletion of max of the heap in (a). Deletion is performed by replacing the root's key with the key at the last leaf and then recovering the heap property by repeatedly exchanging keys with children.

Basic Heap Operations

A max-heap supports $\mathcal{O}(\log n)$ **insertions**, $\mathcal{O}(1)$ time **lookup** for the **max** element, and $\mathcal{O}(\log n)$ **deletion** of the **max** element. Searching for arbitrary keys needs $\mathcal{O}(n)$ time.

- Use a **heap** when all you care about is the **largest** or **smallest elements**, and you do not need to support fast lookup, delete, or search operations for arbitrary elements.
- A heap is a good choice when you need to compute the **k largest** or **k smallest elements** in a collection. For the **former**, use a **min-heap**, for the **latter**, use a **max-heap**.

Heap implementations

In C++, you can use `std::priority_queue`. This is a max-heap by default:

```
1 #include <queue>
2 int main() {
3     std::priority_queue<int> max_heap;
4 }
5
```

A min-heap for integers is defined as follows:

```
1 #include <queue>
2 int main() {
3     // counter-intuitive: have to use the > comparator
4     std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap;
5 }
6
```

https://en.cppreference.com/w/cpp/container/priority_queue

Heap implementations

In python, you can use `queue.PriorityQueue`. This is a min-heap by default:

```
1 import queue
2 minheap = queue.PriorityQueue()
3
```

A (hacky) way to implement a max-heap is to overload the `>` operator:

```
1 import queue
2 class Data:
3     def __init__(self, value):
4         self.value = value
5     def __gt__(self, other):
6         return self.value < other.value
7 minheap = queue.PriorityQueue()
8 minheap.put(Data(4))
9
```

<https://docs.python.org/3/library/queue.html>

Find the k Longest Words in a Stream

Suppose you were asked to write a program which takes a **sequence of strings** presented as a **stream**: you cannot back up to read an earlier value. Your program must compute the **k longest strings** in the sequence.

Find the k Longest Words in a Stream (Solution)

k_longest_words.cpp words.txt

```
1 // Each string is processed in  $O(\log k)$  time, which is the time to add and to remove the
2 // minimum element from the heap. Therefore, if there are  $n$  strings in the input, the
3 // time complexity to process all of them is  $O(n \log k)$ .
4 int main() {
5     string path = "words.txt";
6     ifstream ifs{path}; // open file as a stream
7     if (ifs.fail()) { cerr << "cannot find: " << path << endl; }
8     auto comparator = [](const string &a, const string &b) {
9         return a.size() > b.size(); // comparisons are unintuitive the other way round for heaps
10    };
11    priority_queue<string, vector<string>, decltype(comparator)> min_heap(comparator);
12    size_t k = 7; // k longest words
13    string word;
14    while (ifs >> word) { // extract string from stream into a string variable word
15        // We could improve best-case time complexity by first comparing the word's
16        // length with the length of the string at the top of the heap (getting this string takes
17        //  $O(1)$  time) and skipping the insert if the word is shorter.
18        min_heap.push(word);
19        if (min_heap.size() > k) { min_heap.pop(); } // remove the shortest string
20    }
21    while (!min_heap.empty()) {
22        cout << min_heap.top() << " : " << min_heap.top().size() << endl;
23        min_heap.pop(); // remove min element from heap
24    }
25 }
```

Sorting

Merge Sorted Files

This problem is motivated by the following scenario. You are given 500 files, each containing stock trade information. Each trade is encoded by a line in the following format:
1232111,AAPL,30,456.12. The first number is the time of the trade expressed as the number of milliseconds since the start of the day's trading. Lines within each file are sorted in increasing order of time. The remaining values are the stock symbol, number of shares, and price. You are to create a single file containing all the trades from the 500 files, sorted in order of increasing trade times. The individual files are of the order of 5-100 megabytes; the combined file will be of the order of five gigabytes.

Merge Sorted Files

This problem is motivated by the following scenario. You are given 500 files, each containing stock trade information. Each trade is encoded by a line in the following format:
1232111,AAPL,30,456.12. The first number is the time of the trade expressed as the number of milliseconds since the start of the day's trading. Lines within each file are sorted in increasing order of time. The remaining values are the stock symbol, number of shares, and price. You are to create a single file containing all the trades from the 500 files, sorted in order of increasing trade times. The individual files are of the order of 5-100 megabytes; the combined file will be of the order of five gigabytes.

In conclusion, we are trying to solve the **following problem**:

Write a program that takes as **input a set of sorted sequences** and **computes the union of these sequences as a sorted sequence**. For example, if the input is $\langle 3, 5, 7 \rangle$, $\langle 0, 6 \rangle$, and $\langle 0, 6, 28 \rangle$, then the output is $\langle 0, 0, 3, 5, 6, 6, 7, 28 \rangle$.

Merge Sorted Arrays Code (1/2)

merge_sorted_arrays.cpp

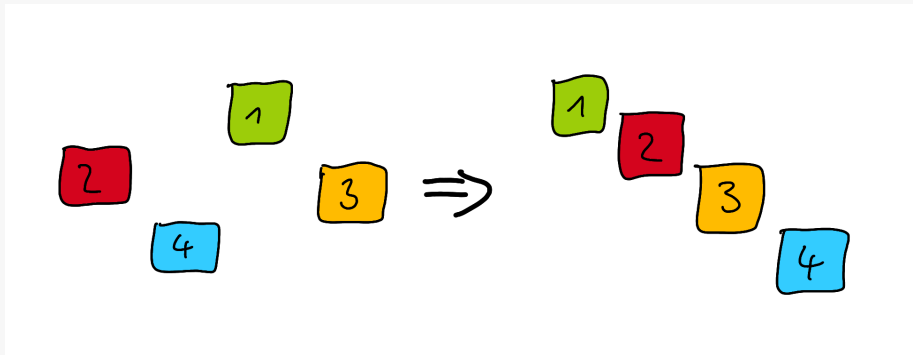
```
1 struct Array_Info { // we'll make a min heap with Array Info elements
2     vector<int> *vec;
3     size_t index; // current index
4     int current_number() const { return (*vec)[index]; }
5 };
6
7 // Let k be the number of input sequences. Then there are no more than k elements in the
8 // min-heap. Both extract-min and insert take  $O(\log k)$  time. Hence, we can do the merge in
9 //  $O(n \log k)$  time. The space complexity is  $O(k)$  beyond the space needed to write the final
10 // result. In particular, if the data comes from files and is written to a file, instead of
11 // arrays, we would need only  $O(k)$  additional storage.
```

Merge Sorted Arrays Code (2/2)

merge_sorted_arrays.cpp

```
1 vector<int> merge_sorted_arrays(vector<vector<int>> &sorted_arrays) {
2     auto comparator = [](const Array_Info &a, const Array_Info &b) {
3         return a.current_number() > b.current_number(); // comparison for min heap
4     };
5     priority_queue<Array_Info, vector<Array_Info>, decltype(comparator)> min_heap(comparator);
6     for (vector<int> &sorted_array : sorted_arrays)
7         if (!sorted_array.empty()) // push on heap only non empty vectors
8             min_heap.push(Array_Info{&sorted_array, 0});
9
10    vector<int> result;
11    while (!min_heap.empty()) {
12        Array_Info smallest_array = min_heap.top(); // get array with next smallest number
13        min_heap.pop();
14        result.push_back(smallest_array.current_number()); // insert number into result vector
15        // increment index, if array still has unprocessed numbers push the next to the min heap
16        if (++smallest_array.index < smallest_array.vec->size())
17            min_heap.push(smallest_array);
18    }
19    return result;
20 }
```

Sorting Basics

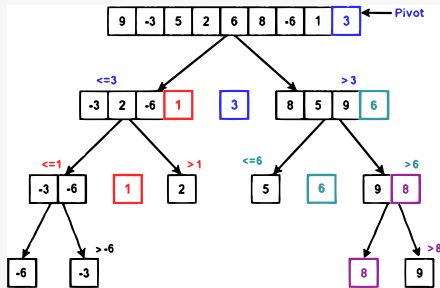


Sorting is used to **preprocess** collections to **make searching faster** (binary search), as well as **identify items that are similar** (for example, students are sorted on test scores).

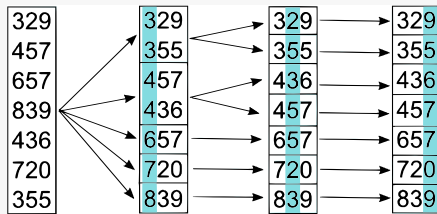
→ Many problems are **easier or more efficient** to solve if we sort the data first.

Sorting Basics

Most **sorting routines** are based on **comparisons** that takes two items a and b and returns $a < b$ as a boolean. They can sort in $\mathcal{O}(n \log n)$. However, it is also possible to use **numerical attributes**, like in **radix sort** ($\mathcal{O}(nk)$) and some cases of **bucket sort** ($\mathcal{O}(nb)$).



Quicksort



Radix Sort

Sorting in $\mathcal{O}(n)$

For **specialized input**, for example, a very **small range of values**, or a **small number of values**, it's possible to sort in $\mathcal{O}(n)$ time rather than $\mathcal{O}(n \log n)$ time.

For example, suppose you are asked the following question: Given a very large array of **Person objects**, sort the people in **increasing order of age**.

Sorting in $\mathcal{O}(n)$

For **specialized input**, for example, a very **small range of values**, or a **small number of values**, it's possible to sort in $\mathcal{O}(n)$ time rather than $\mathcal{O}(n \log n)$ time.

For example, suppose you are asked the following question: Given a very large array of **Person objects**, sort the people in **increasing order of age**.

We're given **two interesting bits of knowledge** here:

1. It's a **large array**, so **efficiency** is key.
2. We are **sorting people by age**, so we know the values are in a **small range** ($0 \leq \text{age} < 130$).

Sorting in $\mathcal{O}(n)$

For **specialized input**, for example, a very **small range of values**, or a **small number of values**, it's possible to sort in $\mathcal{O}(n)$ time rather than $\mathcal{O}(n \log n)$ time.

For example, suppose you are asked the following question: Given a very large array of **Person objects**, sort the people in **increasing order of age**.

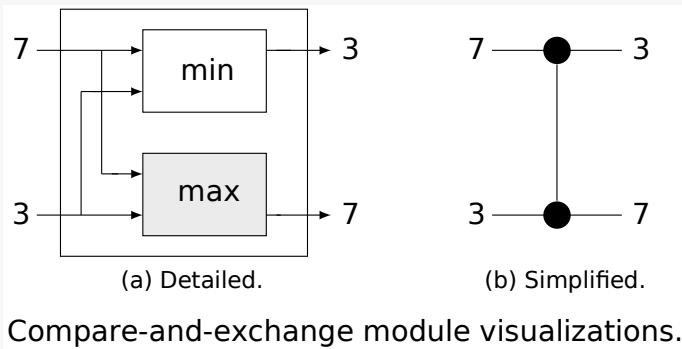
We're given **two interesting bits of knowledge** here:

1. It's a **large array**, so **efficiency** is key.
2. We are **sorting people by age**, so we know the values are in a **small range** ($0 \leq \text{age} < 130$).

By scanning through the various sorting algorithms, we might notice that **bucket sort** (or radix sort) would be a perfect candidate for this algorithm. In fact, we can **make the buckets small** (just 1 year each) and get $\mathcal{O}(n)$ running time. (If the keys are evenly distributed, the bucket / radix sorting can be easily parallelized.)

Sorting Networks Basics

Compare-and-exchange (COEX) modules are the **building blocks of sorting networks**.



Sorting Networks Basics

In a **sorting network**, the **COEX modules are combined** in such a way that they **always sort a fixed-length sequence of values**. The amount of modules and their execution order are fixed for a network and do not depend on the input values. **Sorting networks** are therefore **data-oblivious algorithms**. → **Sorting networks** can be **hardcoded**.

Sorting Networks Basics

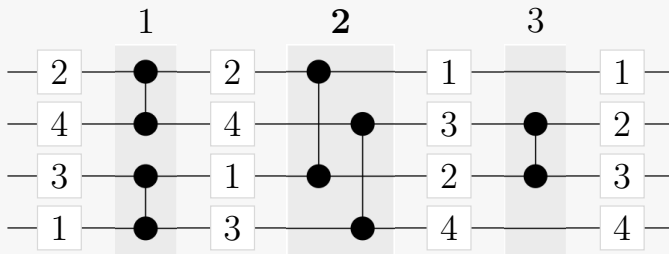
In a **sorting network**, the **COEX modules are combined** in such a way that they **always sort a fixed-length sequence of values**. The amount of modules and their execution order are fixed for a network and do not depend on the input values. **Sorting networks** are therefore **data-oblivious algorithms**. → **Sorting networks** can be **hardcoded**.

Efficient **sorting networks** have an **asymptotic complexity** of $\mathcal{O}(n \log^2 n)$, where n is the number of elements to sort.

→ For **small arrays** that's ok.

Sorting Small Arrays With Sorting Networks

Here, you see an **example** of a **sorting network** for **four elements**. The input comes from the left and runs to the right through the network. This sorting network contains **5 COEX modules**. The numbers above the gray boxes index the **parallel steps**. Each parallel step is characterized by the fact that there are no dependencies between individual modules. All COEX modules within a gray box can thus be executed simultaneously.



Sorting Small Arrays With Sorting Networks

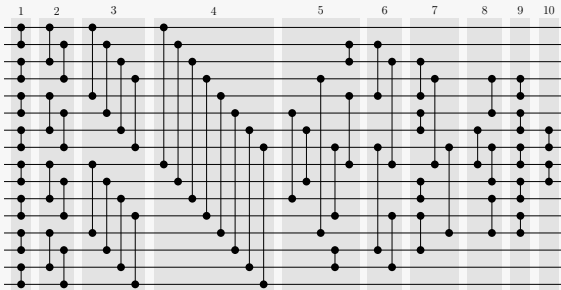
Same sorting network hardcoded. `sort_4_int.cpp` 

```
1 // macro simulates a COEX module
2 #define COEX(a, b) if(a > b) {int c = a; a = b; b = c;}
3 // sorting network to sort 4 integers
4 inline void sort_4_int(int *arr) {
5     COEX(arr[0], arr[1]); COEX(arr[2], arr[3]); // step 1
6     COEX(arr[0], arr[2]); COEX(arr[1], arr[3]); // step 2
7     COEX(arr[1], arr[2]);                       // step 3
8 }
```

A bigger Sorting Network

Sorting networks can be efficiently implemented with **vector instructions**.

See `vectorized_sorting_networks.tar.gz` 



This network sorts **16 elements** with **60 COEX modules**. Source:

<https://www.hanshq.net/knuth-cheque.html>



There are also **median networks** to compute the median in a data-oblivious, hardcoded fashion.

Compute the Intersection of Two Sorted Arrays

Write a program which takes as **input two sorted arrays**, and **returns** a **new array** containing **elements** that are present in **both of the input arrays**, ignoring duplicates. For example, if the inputs are $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, the output should be $\langle 5, 6, 8 \rangle$.



Compute the Intersection of Two Sorted Arrays

Write a program which takes as **input two sorted arrays**, and **returns a new array** containing **elements** that are present in **both of the input arrays**, ignoring duplicates. For example, if the inputs are $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, the output should be $\langle 5, 6, 8 \rangle$.

intersect_sorted_arrays.cpp 

```
1 // Since we spend  $O(1)$  time per input array element,  
2 // the time complexity for the entire algorithm is  $O(m + n)$ .  
3 vector<int> intersect_sorted_arrays(const vector<int> &A, const vector<int> &B) {  
4     vector<int> intersection_A_B;  
5     size_t i = 0, j = 0;  
6     while (i < A.size() && j < B.size()) {  
7         if (A[i] == B[j] && (i == 0 || A[i] != A[i - 1])) {  
8             intersection_A_B.emplace_back(A[i]);  
9             ++i, ++j;  
10        } else if (A[i] < B[j]) {  
11            ++i;  
12        } else { // A[i] > B[j]  
13            ++j;  
14        }  
15    }  
16    return intersection_A_B;  
17 }
```

Compute the Intersection of Two Sorted Arrays

Write a program which takes as **input two sorted arrays**, and **returns** a **new array** containing **elements** that are present in **both of the input arrays**, ignoring duplicates. For example, if the inputs are $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, the output should be $\langle 5, 6, 8 \rangle$.

Compute the Intersection of Two Sorted Arrays

Write a program which takes as **input two sorted arrays**, and **returns** a **new array** containing **elements** that are present in **both of the input arrays**, ignoring duplicates. For example, if the inputs are $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, the output should be $\langle 5, 6, 8 \rangle$.

What would you do if **one array was much larger** than the other?

Compute the Intersection of Two Sorted Arrays

Write a program which takes as **input two sorted arrays**, and **returns a new array** containing **elements** that are present in **both of the input arrays**, ignoring duplicates. For example, if the inputs are $\langle 2, 3, 3, 5, 5, 6, 7, 7, 8, 12 \rangle$ and $\langle 5, 5, 6, 8, 8, 9, 10, 10 \rangle$, the output should be $\langle 5, 6, 8 \rangle$.

What would you do if **one array was much larger** than the other?

For each element in the small array, perform a **binary search** in the large array.

This runs in $\mathcal{O}(m \cdot \log(n))$ (where m is the size of the small array, and n is the size of the large array), in contrast to $\mathcal{O}(m + n)$.

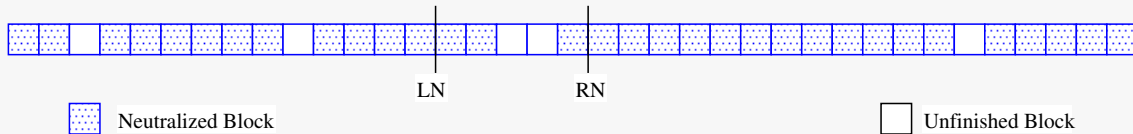
For example, let $m = 10$ and $n = 1024$. Then the binary search approach runs in $\text{ca. } 10 \cdot \log_2 1024 = 100$ steps, while the full walkthrough takes $\text{ca. } 1034$ steps.

Parallel Sorting

Most sorting algorithms can be efficiently parallelized. Although the techniques for parallelizing quicksort, mergesort and distribution-based sorting routines are well known, they are challenging to implement.

Fortunately there are **free implementations** of fast parallel general purpose sorting algorithms, for example <https://github.com/SaschaWitt/ips4o>.

Efficient **parallelization** of **quicksort**: <http://www.cse.chalmers.se/~tsigas/papers/Pquick.pdf>

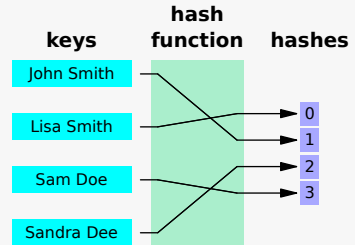
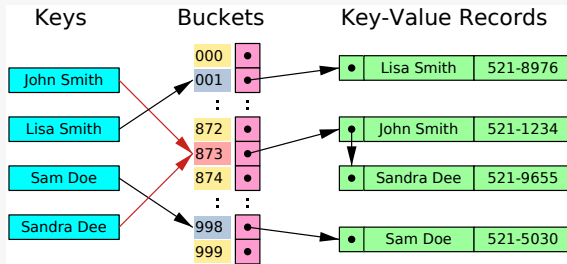


With **this technique** you can **even parallelize quickselect** (find the k -th smallest element in an unordered array, for example, the median).

Hash tables

Hash Table Basics

The idea of a **hash table** is that **data**, whatever its type, can be fed to a hash function that reduces it to an **integer** called 'hash'. The hash becomes an **array index** that leads directly to a table entry. If the table entry then matches the data, the search is a success. If the hash always leads directly to the table entry, the hash table allows access in **constant time**.



Hash Table Basics

Properties of hash functions:

- A hash that creates a table in which there are **no collisions** for a given set of keys is called a **perfect hash**.
- A hash that creates a table with **no unused spaces** is called a **minimal hash**.

The **Holy Grail** of hash functions is a **minimal perfect hash**, creating a table with **no collisions** and **no empty spaces**.

Minimal Perfect Hash Example

minimal_perfect_hash.cpp 

```
1 struct kv { // (key,value) pairs
2     char const *key;
3     unsigned int value;
4 };
5
6 // the 26 valid entries each begin with a different letter and they are sorted,
7 // so a hash based on the first letter is a perfect minimal hash
8 static kv table[] = { // this is the NATO phonetic alphabet
9     {"alpha", 1}, {"bravo", 2}, {"charlie", 3}, {"delta", 4}, {"echo", 5},
10    {"foxtrot", 6}, {"golf", 7}, {"hotel", 8}, {"india", 9}, {"juliet", 10},
11    {"kilo", 11}, {"lima", 12}, {"mike", 13}, {"november", 14}, {"oscar", 15},
12    {"papa", 16}, {"quebec", 17}, {"romeo", 18}, {"sierra", 19}, {"tango", 20},
13    {"uniform", 21}, {"victor", 22}, {"whiskey", 23}, {"xray", 24}, {"yankee", 25},
14    {"zulu", 26}};
15
16 unsigned int my_hash(char const *key) {
17     // Prevent accessing undefined storage in case key is something like "@#$$".
18     if (key[0] < 'a' || key[0] > 'z')
19         return 0;
20     return (key[0] - 'a');
21 }
22
23 kv *find_hash(kv *first, kv *last, char const *key) {
24     unsigned int i = my_hash(key); // you don't need to call strcmp if (key[0] < 'a' || key[0] > 'z')
25     return strcmp(first[i].key, key) ? last : first + i; // strcmp returns 0 if C strings are equal
26 }
```

A Simple Perfect Hash for Strings

Sequence alignment in Bioinformatics is the task of reconstructing a full DNA-sequence from partial and erroneous fragments. Current technology for DNA sequencing cannot read the full DNA sequence at once, but **only smaller parts (reads), and often produces errors**. We want to figure out how the DNA-reads overlap to reconstruct the full sequence.

DeBruijn-Graph

One possibility is via a **DeBruijn-Graph**, which is constructed from the reads. For each read, all substrings of a fixed length k are constructed, and if two substrings appear consecutive in a read, they are connected by an edge.

READS:

ACGT
CGTC

DeBruijn-Graph:
($k=3$)

ACG - CGT - GTC

Example

(Unknown) correct sequence:

ATCTAAGTG CAT

Reads from
sequencing machine:

ATTTAAT

AAGGGC

CTAAGT

GTGCAT

TCTAAG

TTAGTG

Example

(Unknown) correct sequence:

ATCTAAGTG CAT

Reads from
sequencing machine:

AT~~X~~TAA~~X~~
AA~~G~~GC
CTAAGT
GTGCAT
TCTAAG
T~~X~~AGTG

Correct assembly:

ATCTAAGTG CAT

Example

CONSTRUCTION OF THE DEBRUYN-GRAPH
ATTTAAT

ATT → TTT → TTA → TAA → AAT

Example

CONSTRUCTION OF THE DEBRUYN-GRAPH
A A G G G C

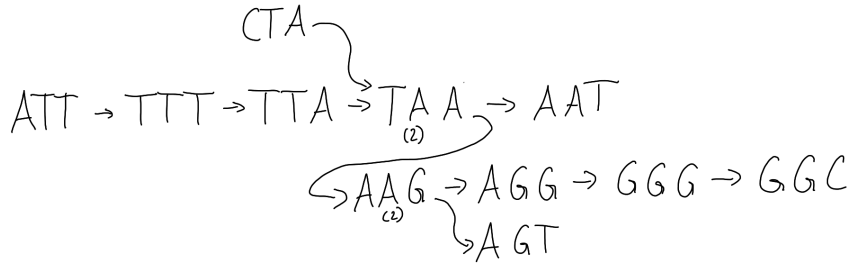
ATT → TTT → TTA → TAA → AAT

AAG → AGG → GGG → GGC

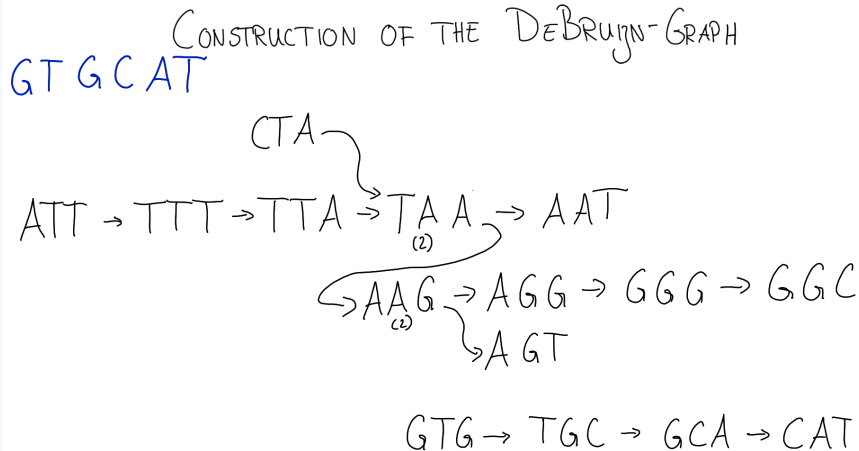
Example

CONSTRUCTION OF THE DEBRUYN-GRAPH

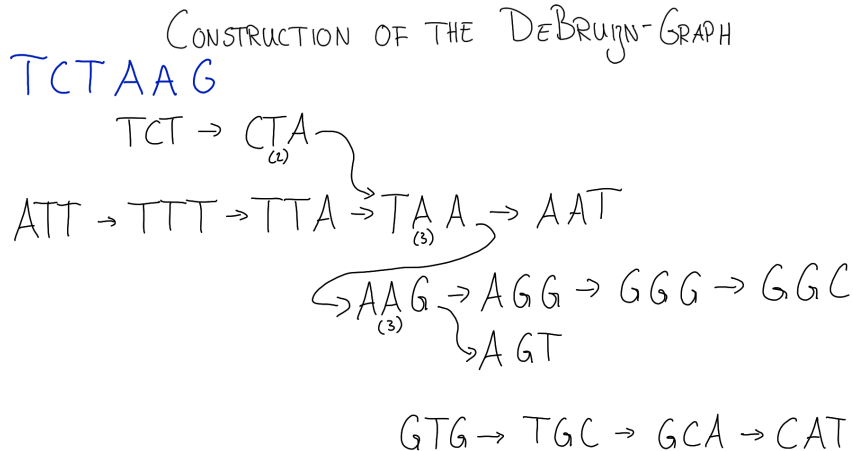
CTA A GT



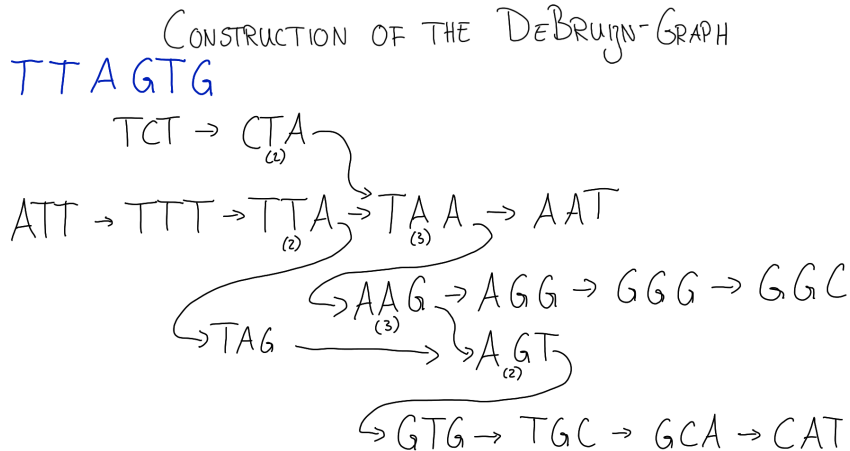
Example



Example

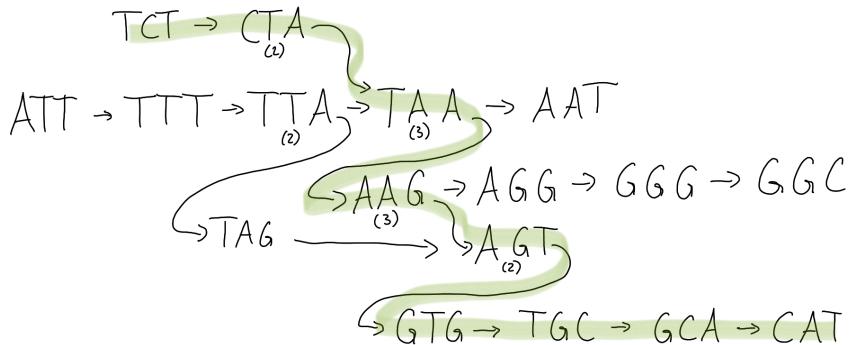


Example



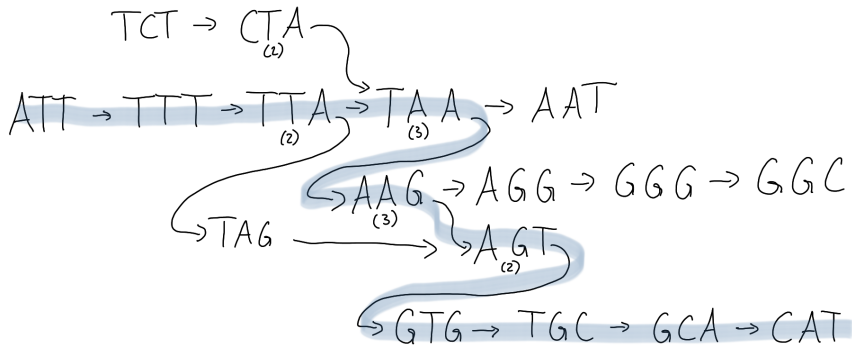
Example

CONSTRUCTION OF THE DEBRUYN-GRAPH



Example

CONSTRUCTION OF THE DEBRUYN-GRAPH



Example

Two Possible Reconstructions:

X T C T A A G T G C A T
A T T ~~T~~ A A G T G C A T

(1 missing)

(1 error)

A T C T A A G T G C A T

Efficient Construction of DeBruijn-Graph Using Hashes

When constructing the DeBruijn-Graph, we need to check repeatedly if a substring is already present in our graph. We can implement this efficiently using a hash function that maps DNA-substrings to positive integers:

```
1  uint32_t hash_DNA_fixedlength_ifelse ( string dna_string , uint32_t length){
2      uint32_t hash = 0;
3      for ( uint32_t i=0; i<length; i++){
4          if ( dna_string[i] == 'A')
5              hash += 0;
6          if ( dna_string[i] == 'C')
7              hash += pow(4,i);
8          if ( dna_string[i] == 'G')
9              hash += 2*pow(4,i);
10         if ( dna_string[i] == 'T')
11             hash += 3*pow(4,i);
12     }
13     return hash;
14 }
15
```

Exercises (Upload Solutions to your Repository)

1. Find the k brightest stars in a stream.



`find_k_brightest_stars.cpp` → `stars.txt` →

2. Sort employees by age with bucket sort.

`bucket_sort.cpp` →

3. Implement Green's sorting network.

`sort_16_int.cpp` →

4. Write a minimal perfect hash function.

`my_hash.cpp` →

5. Find the nearest repeated entries in an array.

`find_nearest_repeated.cpp` →

Exam Questions

- **Describe briefly** the **heap** data structure.
- **How** would you **find the k longest words** in a **data stream**? (You cannot back up to read an earlier value.)
- **How** would you address the problem of **merging multiple sorted files** that are **too large for RAM**?
- What are **sorting networks**?
- **Why sorting** may speed up **set operations**?
- **What is a minimal perfect hash**?